

Risk Management

Funzioni implementate per la risoluzione degli esercizi

Implementiamo una funzione "PtoR" che trasforma una matrice di prezzi in una matrice di rendimenti netti, una funzione "mu" che ne calcola la media e una funzione "Sigma" che ne calcola la varianza/covarianza.

```
PtoR <- function(P,
                 n = 1)

# n = 1 rendimento semplice
# n = c rendimento continuamente capitalizzato

{
  P <- as.matrix(P)
  P2 <- P[2:nrow(P),]
  P1 <- P[1:(nrow(P) - 1),]
  if (n == "c") log(P2/P1) else ( (P2/P1)^(1/n) - 1 ) * n
}

#-----
mu <- function(X)

{
  if (is.list(X)) as.vector( X[[1]]) else colMeans(X)
}

#-----
Sigma <- function(X)

{
  if (is.list(X)) as.matrix( X[[2]]) else cov(X)
}
```

La funzione "portfolio" identifica il portafoglio efficiente in corrispondenza a un certo parametro c :

$$\alpha = \frac{\Sigma^{-1} m \mathbf{1}_N^T \Sigma^{-1} m}{\mathbf{1}_N^T \Sigma^{-1} m} \eta_c + \frac{\Sigma^{-1} \mathbf{1}_N}{\mathbf{1}_N^T \Sigma^{-1} \mathbf{1}_N} (1 - \eta_c) = \alpha_M \eta_c + \alpha_{\min} (1 - \eta_c).$$

```
portfolio <- function(X,c)

{
  m <- mu(X)
  S <- Sigma(X)
  Sm <- solve(S,m) #S^-1 * m #affinchè le quote sommino a 1
  Se <- solve(S,rep(1,nrow(S))) #S^-1 * 1N
  eta <- sum(Sm) #1N' * S^-1 * m
  aM <- Sm/eta
}
```

```

amin <- Se/sum(Se)  #(S^1 *1N)/(1N' * S^-1 * 1N)
aM * eta/c + amin * (1-eta/c)
#formula della frontiera efficiente
}

```

La funzione "frontier" disegna la frontiera efficiente dato un campione di dati di rendimento e/o una stima iniziale del vettore dei rendimenti attesi e della matrice di varianza/covarianza. Se sd = TRUE, allora sull'asse x verrà rappresentata la deviazione standard invece della varianza.

```

frontier <- function(X,
                     x = NULL, #per fissare la lunghezza dell'asse x
                     x0 = NULL,
                     mesh = 100, #numero di portoflio usati per rappr
                     sd = FALSE, #sull'asse x ho la varianza, non sd
                     graph = TRUE,
                     title = "Efficient frontier",
                     output = FALSE)
{
  S <- Sigma(X)
  m <- mu(X)

  #due portafogli efficienti
  a1 <- portfolio(X,c = 1)
  a2 <- portfolio(X,c = 0.5)

  #le loro medie
  m1 <- as.numeric(a1%*%m)
  m2 <- as.numeric(a2%*%m)

  #var e cov
  s1 <- as.numeric(a1%*%S%*%a1)
  s2 <- as.numeric(a2%*%S%*%a2)
  s12<- as.numeric(a1%*%S%*%a2)
  ss <- diag(S)

  if (is.null(x)) x <- max(ss)

  x <- max(x,ss)

  if (is.null(x0)) x0 <- 0

  #nuovo portafoglio come combinazione convessa dei due portafogli
  #efficienti, sarà efficiente
  A <- s1 + s2 - 2*s12
  B <- s2 - s12
  C <- s2 - 1.2*x
  t1 <- as.numeric((B - sqrt(B^2 - A*C))/A)
  t2 <- as.numeric((B + sqrt(B^2 - A*C))/A)
  t <- t1 + (0:mesh)*(t2 - t1)/mesh
  mt <- t*m1 + (1 - t)*m2
  st <- t^2*s1 + (1 - t)^2*s2 + 2*t*(1 - t)*s12
  xl <- "Var"
}

```

```

if (sd) {
  st <- sqrt(st)
  xl <- "Std. dev."
  x <- sqrt(x)
  x0 <- sqrt(x0)
  ss <- sqrt(ss)}

if (graph){
  plot(st,mt,
        type="l",main=title,
        xlab=xl,ylab="Mean",xlim=c(max(x0,min(st)),max(st)))
  points(ss,m)
}

if (output) cbind(st,mt)
}

```

La funzione "portfolio.target" serve a trovare i pesi di un portfolio con specifiche caratteristiche, coincide con portfolio con "what=c", con "what=mean" impone rendimento uguale al target fissato, con "what=var" impone varianza target, mentre con "what=sr" massimizza Sharpe Ratio. Infine con "what=b" massimizza

$$\mu - b * \sigma^2$$

```

portfolio.target<-function(X,
                           target = NULL,
                           what = c("mean","var","c","b","sr"),
                           r0 = 0,
                           graph = TRUE,
                           title = "Efficient frontier",
                           output = TRUE)

{
  S <- Sigma(X)
  m <- mu(X)

  a1 <- portfolio(X,c=1)
  a2 <- portfolio(X,c=0.5)

  m1 <- as.numeric(a1%%m)
  m2 <- as.numeric(a2%%m)

  s1 <- as.numeric(a1%%S%%a1)
  s2 <- as.numeric(a2%%S%%a2)
  s12 <- as.numeric(a1%%S%%a2)

  if (!is.null(target)) target <- as.numeric(target)

  A <- s1 + s2 - 2*s12
  B <- s2 - s12

```

```

C <- s2

tmin <- B/A
smin <- C - B^2/A

if (what=="mean"){
  t <- (target - m2)/(m1 - m2)
  a <- t*a1 + (1 - t)*a2
  c <- 1/(t + 2*(1 - t))
}

if (what=="var"){

  if (is.null(target)){
    a <- tmin*a1 + (1 - tmin)*a2
    c <- 1/(tmin + 2*(1 - tmin))}

  if (!is.null(target)){
    if (B^2<A*(C - target)) stop("Warning: target too low")
    t <- ( B - sqrt( B^2 - A*(C - target) ) )/A
    a <- t*a1 + (1 - t)*a2
    c <- 1/(t + 2*(1 - t))}
}

if (what=="b"){
  a <- solve(S,m-target)
  c <- sum(a)
  a <- a/c
}

if (what=="c") {
  a <- portfolio(X,c=target)
  c <- target
}

if (what=="sr"){

  if (is.null(target)){
    t <- ( (m1 - m2)*C + (m2 - r0)*B )/( A*(m2 - r0) + B*(m1 - m2) )
    a <- t*a1 + (1 - t)*a2
    c <- 1/(t + 2*(1 - t))
  }

  if (!is.null(target)){
    AA <- (m1 - m2)^2 - A*target^2
    BB <- (m1 - m2)*(m2-r0) + B*target^2
    CC <- (m2 - r0)^2 - C*target^2

    if (BB^2<AA*CC) stop("Warning: target too high")

    t1 <- -(BB + sqrt(BB^2 - AA*CC))/ AA
    t2 <- -(BB - sqrt(BB^2 - AA*CC))/ AA
  }
}

```

```

d1 <- ((t1*(m1 - m2) + m2 - r0) / sqrt(t1^2*A - 2*t1*B + C) - target)^2
d2 <- ((t2*(m1 - m2) + m2 - r0) / sqrt(t2^2*A - 2*t2*B + C) - target)^2

if (d1<d2) t <- t1

if (d2<=d1) t <- t2

a <- t*a1 + (1-t)*a2
c <- 1/(t + 2*(1-t))

if (c<0) stop("Warning: target too low")
}
}

ma <- as.numeric(a%**m)
sa <- as.numeric(a%**S%**a)

if (graph){
  frontier(X,x=sa,title=title,graph=graph)
  points(sa,ma,col="blue",pch=21,bg="red")
  if (c==Inf)
    abline(v = sa,col="magenta")
  else
    abline(a=ma-0.5*c*sa,b=c/2,col="magenta")
}

if (output) list(weights = a,
                 b = ma-c*sa,
                 c = c,
                 mean = ma,
                 var = sa,
                 sr = (ma - r0)/sqrt(sa))
}

```

La funzione "portfolio.opt" calcola il portafoglio ottimo per un investitore data una funzione di utilità $U(W, m, s^2, \dots)$ da massimizzare, dove W indica la ricchezza dell'investitore, m il rendimento atteso del portafoglio, s^2 è la varianza del portafoglio. Osserviamo che se $r_0 = \text{NULL}$ -> l'ottimo viene cercato sulla frontiera efficiente, tra portafogli rischiosi, altrimenti lungo la market capital line.

```

portfolio.opt<-function(U,W,X,...,
                        title = "The optimal portfolio choice",
                        indiff = TRUE,
                        graph = TRUE,
                        r0 = NULL,
                        output = FALSE)

{
  m <- mu(X)
  S <- Sigma(X)

  if (is.null(r0)){
    F <- function(c,...){

```

```

a <- portfolio(X,c)
U(W,a**m,a**S**a,...) }

#ottimizzo la funzione

H <- optimize(f = F,
              interval = c(1e-05,1e+05),
              ...,
              maximum = TRUE)

a <- portfolio.target(X,
                      target = H[[1]],
                      what = "c",
                      0,
                      graph,
                      title,
                      output = TRUE)

if (graph&indiff){

  x0 <- 0.75*a$var
  x1 <- 1.25*a$var

  y0 <- uniroot(function(m)U(W,m,x0,...)-H[[2]],
                interval = c(-1,1) + a$mean,
                extendInt = "yes")[[1]]
  y1 <- uniroot(function(m)U(W,m,x1,...)-H[[2]],
                interval = c(-1,1) + a$mean,
                extendInt = "yes")[[1]]

  x <- seq(x0,x1,length.out = 1000)
  y <- seq(y0,y1,length.out = 1000)
  z <- outer(x,y,
             FUN = function(x,y)U(W,y,x,...) - as.numeric(H[[2]]))
  contour(x,y,z,
          levels = 0,
          col = "green",
          add = TRUE,
          drawlabels = FALSE)
  #curva di indifferenza
}

if (output) Out <- list(weights = a$weights,
                        mean = a$mean,
                        var = a$var,
                        b = a$b,
                        c=H[[1]],
                        maxU=H[[2]])
}

if (!is.null(r0)){
  aSR <- portfolio.target(X,
                          what = "sr",
                          r0 = r0,

```



```

  if (output) Out
}

```

La funzione "co.var" calcola la minima varianza di un portafoglio con un rendimento target, che soddisfi il vincolo

$$M_b * a \geq l_b$$

Osserviamo che se Mb=NULL, è imposto come matrice identità. Se mt=NULL riporta rendimento minimo e massimo ottenibili e i portafogli e varianze corrispondenti.

```

co.var<-function(mt=NULL,
                 X,
                 lb=0,
                 Mb=NULL)

{
  N <- ncol(X)
  m <- mu(X)
  S <- Sigma(X)

  if (is.null(Mb)) Mb <- diag(N)
  if (length(lb)==1) lb <- rep(lb,nrow(Mb))
  if (nrow(Mb)!=length(lb))stop("Warning: Mb and lb do not match")

  f.obj <- m
  f.con <- rbind(rep(1,N),Mb)
  f.dir <- c("=",rep(">=",nrow(Mb)))
  f.rhs <- c(1+2*N,lb+Mb%%rep(2,N))

  H0 <- lp("min",f.obj,f.con,f.dir,f.rhs)
  H1 <- lp("max",f.obj,f.con,f.dir,f.rhs)

  m0 <- H0$objval-2*sum(m)
  a0 <- H0$solution-rep(2,N)
  m1 <- H1$objval-2*sum(m)
  a1 <- H1$solution-rep(2,N)

  if (is.null(mt)) U <- list(a0=a0, m0=m0, s0=a0%%S%%a0,
                             a1=a1, m1=m1, s1=a1%%S%%a1)

  if (!is.null(mt)){

    if (mt>m1|mt<m0) U=list(var=Inf,weights=a0)

    else{
      H <- solve.QP(Dmat = S,
                    dvec = rep(0,N),
                    Amat = cbind( rep(1,N) , mu(X) , t(Mb) ),
                    bvec = c( 1 , mt , lb ),
                    #tutti alpha superiori a un certo lb

                    meq = 2)
      U=list(var=2*H$value,weights=H$solution)}
    }
  }
}

```



```

}
U
}

```

La funzione `co.frontier` disegna la frontiera efficiente sotto il vincolo:

$$M_b * a \geq l_b$$

```

co.frontier<-function(X,
                      x=NULL,
                      x0=NULL,
                      lb=0,
                      Mb=NULL,
                      sd=FALSE,
                      mesh=100,
                      graph = TRUE,
                      output = FALSE)

{
  m <- mu(X)
  S <- Sigma(X)
  N <- ncol(X)

  U <- co.var(NULL,X,lb,Mb)

  T <- U$m0 + (U$m1 - U$m0)*(1:mesh)/(mesh+1)
  #ripeto per ogni valore di T
  V <- (1:mesh)

  for (n in seq(T))  V[n] <- co.var(mt=T[n],X,lb,Mb)$var

  V <- c(U$s0,V,U$s1)
  T <- c(U$m0,T,U$m1)

  if (sd) V <- sqrt(V)

  if (graph==TRUE){
    frontier(X,
             x,
             x0,
             sd=sd,
             title = "Efficient frontier with and without constraints")

    lines(V,T,col="red")

    points(cbind(V[c(1,102)],T[c(1,102)]),
           col="blue",pch=21,bg="green")
  }

  if (output==TRUE)list(data = cbind(V,T),a0 = U$a0,a1 = U$a1)
}

```

Questa funzione calcola il Value at Risk (VaR) al livello p stimandolo a partire solo dalle perdite più estreme dei rendimenti, usando la teoria dei valori estremi (EVT) e assumendo che la coda sinistra

della distribuzione segua una legge di Pareto.

```
#-----
q_EVT<-function(p,X,u = NULL)
#quantile di ordine p, EVT
{
  if (is.null(u)) u <- as.numeric(quantile(-X,0.95,names=FALSE))
  #prende il 95° percentile delle perdite se u non è dato
  #oltre a questo valore: 5% delle perdite peggiori, oltre
  #questo valore considero le perdite come valori estremi
  T <- length(X)
  Tu <- length(X[X<=-u]) #rendimenti estremi
  xi <- mean(log(-X[X<=-u]/u)) #stima di Hill
  #+ è grande, + le perdite estreme sono probabili
  p[p>Tu/T] <- Tu/T
  -u*( (Tu/T)/p )^xi
}
#X:vettore dei rendimenti
#p: livello del quantile (0.01 per es)
#u:soglia
```

La funzione d_EVT calcola la densità condizionale su

$$X \leq u$$

```
d_EVT<-function(t,X,u = NULL)
{
  if (is.null(u)) u <- as.numeric(quantile(-X,0.95,names=FALSE))
  T <- length(X)
  Tu <- length(X[X<=-u])
  xi <- mean(log(-X[X<=-u]/u))

  if (t<=-u) -(1/(t*xi))*(-t/u)^(-1/xi) else 0
}
```

La funzione "p_EVT" calcola la densità cumulativa condizionale su

$$X \leq u$$

```
p_EVT<-function(t,X,u = NULL)
{
  if (is.null(u)) u <- as.numeric(quantile(-X,0.95,names=FALSE))
  T <- length(X)
  Tu <- length(X[X<=-u])
  xi <- mean(log(-X[X<=-u]/u))
  if (t<=-u) (-t/u)^(-1/xi) else 1
}
```

Quando la distribuzione dei rendimenti non è normale:

```

q_CF<-function(p,z1,z2)

{
  F <- qnorm(p)
  #assumo normalità

  F + (z1/6)*(F^2 - 1) + (z2/24)*(F^3 - 3*F) - (z1^2/36)*(2*F^3 - 5*F)
  #tengo conto di coda e asimmetria
}

```

“VaR” calcola il VaR usando la densità Df (distribuzione parametrica ottenuta da “distr”), la funzione quantile Qf o i rendimenti R.

Per l’approccio EVT, si usa la funzione quantile con m e sigma preassegnati, oppure calcolati dalla distribuzione stessa se posti =NULL.

```

VaR<-function(p,
              Df = NULL,
              Qf = NULL,
              R = NULL,
              lambda = NULL,
              m = NULL,
              sigma = NULL,
              ...)

{
  F<-function(p){

    if (!is.null(Df)){
      X <- Df(...)
      mX <- E(X)
      sX <- sd(X)
      if (is.null(m)) m <- mX
      if (is.null(sigma)) sigma <- sX
      V <- -( (q.l(X)(p) - mX)/sX * sigma + m )
    }

    if (!is.null(Qf)){

      mu <- integrate(f = function(p)Qf(p,...),
                     lower = 0,
                     upper = 1)[[1]]
      m2 <- integrate(f = function(p)Qf(p,...)^2,
                     lower = 0,
                     upper = 1)[[1]]
      sd <- sqrt(m2 - mu^2)

      if (is.null(m)) m <- mu
      if (is.null(sigma)) sigma <- sd

      V <- -( (Qf(p,...) - mu) / ( sd ) * sigma + m )
    }
  }
}

```

```

if (!is.null(R)){

  if(is.null(lambda)) lambda <- 1

  T <- length(R)
  w <- lambda^(T:1)
  w <- w/sum(w)
  e <- order(R)
  R <- R[e]
  w <- w[e]
  F <- cumsum(w)
  V <- -min(R[F>=p])
}

list(prob = p, VaR = V)
}
V<-Vectorize(F)
V(p)
}

```

La funzione CVaR serve a calcolare la perdita media.

```

#-----
CVaR<-function(p,
               x = NULL,
               Df = NULL,
               Qf = NULL,
               R = NULL,
               lambda = NULL,
               m = NULL,
               sigma = NULL,
               ...)
#-----
# x è il CVaR threshold; se x= NULL è posto uguale al VaR corrispondente.
{
  CV<-function(x){

    if (!is.null(Df)){
      X <- Df(...)
      mX <- E(X)
      sX <- sd(X)
      if (is.null(m)) m <- mX
      if (is.null(sigma)) sigma <- sX
      X <- ( (X - mX)/sX ) * sigma + m
      ff<-function(y) y * (y<=-x)
      V <- -E(X,ff)/p(X)(-x)
    }

    if (!is.null(Qf)){
      ffun<-function(u)Qf(u,...)
      mu <- integrate(f=ffun, lower=0,upper=1)[[1]]
      m2 <- integrate(function(p)ffun(p)^2,lower=0,upper=1)[[1]]
      sd <- sqrt(m2-mu^2)
      if (is.null(m)) m <- mu
    }
  }
}

```

```

if (is.null(sigma)) sigma <- sd

fun<-function(u) ( (ffun(u) - mu) / ( sd ) ) * sigma + m

up <- uniroot(function(u)fun(u)+x,
              interval=c(1e-05,1-1e-05),
              extendInt="yes")[[1]]
V <- -integrate(f=fun,lower=0,upper=up)[[1]]/up
}

if (!is.null(R)){

  if(is.null(lambda)) lambda <- 1

  T <- length(R)
  w <- lambda^(T:1)
  w <- w/sum(w)
  V <- -R[R<=-x]%*%w[R<=-x]/sum(w[R<=-x])
}

list(level=x,CVaR=V)
}
CVec<-Vectorize(CV)

if (is.null(x)) x<-as.numeric(VaR(p,Df, Qf, R, lambda,
                                m, sigma,...)[2,])
CVec(x)
}

```

Per tenere conto di tutta la distribuzione delle perdite, non solo della coda:

```

EVaR <- function(p,
                Df = NULL,
                Qf = NULL,
                R = NULL,
                lambda=NULL,
                m = NULL,
                sigma = NULL,
                ...)
{
  fpos <- function(y)y*(y>0)

  Hp<-function(t,p){
    if(!is.null(Df)){
      X <- Df(...)
      if (is.null(m)) m <- E(X)
      if (is.null(sigma)) sigma <- sd( X )

      X <- ( (X-E(X))/sd( X ) ) * sigma + m

      h <- p*E(X - t,fpos)-(1- p)*E(t - X,fpos)
    }
  }
}

```

```

if(!is.null(Qf)){
  FFUN<-function(u)Qf(u,...)

  mu <- integrate(f=FFUN, lower=0,upper=1)[[1]]
  m2 <- integrate(function(p)FFUN(p)^2,lower=0,upper=1)[[1]]
  sd <- sqrt(m2 - mu^2)

  if (is.null(m)) m <- mu
  if (is.null(sigma)) sigma <- sd

  FUN<-function(u) ( (FFUN(u) - mu) / ( sd ) ) * sigma + m

  fp<-function(u) fpos( FUN(u) - t )
  fn<-function(u) fpos( t - FUN(u) )

  hp <- integrate(fp,
                  lower=0,upper=1,
                  stop.on.error = FALSE)[[1]]
  hn <- integrate(FUN,
                  lower=0,upper=1,
                  stop.on.error=FALSE)[[1]]
  h <- p * hp - (1 - p) * ( hp + t - hn )
}

if (!is.null(R)){
  if (is.null(lambda)) lambda <- 1
  w <- lambda^(length(R):1)
  w <- w/sum(w)
  h <- p*(as.vector(fpos(R - t))%*%w) - (1 - p)*(as.vector(fpos(t - R))%*%w)
}

h
}

Esc<-function(p){
  e<-uniroot(function(t)Hp(t,p),
             interval=c(-1,1),
             extendInt="yes")[[1]]
  list(p=p,EVaR=-e)
}

EV<-Vectorize(Esc)
EV(p)
}

```